

Algorithm for File Updates in Python

Managing IP Access Control | Google Cybersecurity Certificate

PROJECT DESCRIPTION

My role required keeping an IP address allow list current for a restricted health care subnetwork. Employees working with patient records were granted or revoked access based on their IP, and a separate remove list tracked addresses that needed to come off. I built a Python algorithm that opens the allow list file, reads and parses its contents, removes any flagged IPs, and rewrites the updated list back to the file — replacing a manual process with something that runs consistently every time.

OPEN THE FILE THAT CONTAINS THE ALLOW LIST

I stored the file name in `import_file` to keep it flexible, then used a `with` statement with `open()` in read mode (`"r"`). The `with` block closes the file automatically when the indented section finishes.

```
import_file = "allow_list.txt"

with open(import_file, "r") as file:
```

READ THE FILE CONTENTS

Inside the `with` block, `.read()` pulls the full contents into memory as one string and stores it in `ip_addresses`.

```
with open(import_file, "r") as file:
    ip_addresses = file.read()
```

CONVERT THE STRING INTO A LIST

To target individual IPs I applied `.split()` to `ip_addresses`, which splits at every whitespace and returns a list. The result is reassigned back to `ip_addresses`.

```
ip_addresses = ip_addresses.split()
```

ITERATE THROUGH THE REMOVE LIST

A `for` loop steps through each IP in `remove_list` one at a time, holding the current address in `element` on each pass.

```
for element in remove_list:
    ...
```

REMOVE IP ADDRESSES THAT ARE ON THE REMOVE LIST

An `if` statement checks whether `element` exists in `ip_addresses` using the `in` operator. If it does, `.remove()` pulls it out. The conditional guards against a `ValueError` if the item isn't present. Since the allow list has no duplicates, each removal hits exactly one entry.

```
for element in remove_list:
    if element in ip_addresses:
        ip_addresses.remove(element)
```

UPDATE THE FILE WITH THE REVISED LIST OF IP ADDRESSES

`.join()` called on `"\n"` converts the list back to a string with each IP on its own line. A second `with` statement opens the file in write mode (`"w"`) and `file.write()` drops in the updated string.

```
ip_addresses = "\n".join(ip_addresses)

with open(import_file, "w") as file:
    file.write(ip_addresses)
```

SUMMARY

I built a Python algorithm to automate IP access control for a health care environment. It opens `allow_list.txt` in read mode, pulls the contents into a string with `.read()`, then converts to a list with `.split()` so individual addresses can be targeted. A `for` loop steps through the remove list, and a conditional with `.remove()` clears any address that appears in both lists. The updated list is rejoined with newline separators and written back in write mode. Revoked access gets cleaned up every time the script runs without anyone touching the file manually.